

THE DS4 EQUALIZER

LAB 1

SECTION:

H

SUBMITTED BY:

Simon Samuel

SUBMISSION DATE:

10/31/2025

Problem

In this lab the goal was to create a program that would visualize pitch and roll movement data from a DualShock 4 controller. It would need to display these different kinds of movements via a scrolling bar graph on the terminal. The bar graph aids in showing the controller's real life orientation using text characters. The person manning the controller also should be able to switch between which type of movement it is displaying, roll or pitch, when a button is pressed. It should be able to switch when the Triangle and X buttons are pressed, and it should also stop when the Square button is pressed. Though, for extra credit, we could make it switch using only one button. Another limitation of this lab was we could only use one `scanf()` and one `printf()` throughout the entire program.

Analysis

The main input values of this lab were the gyroscope vector values from the controller, as well as the 4 button inputs, though the only ones we would need were the Square, Triangle, and X. We were able to read these values using the `ds4rd.exe` program we were already provided, but with the `-g` and the `-b` flags. The desired output of the program was a continually updating bar graph, that represented the pitch or roll orientation of the controller. Roll is the side-to-side tilt, while pitch is the front to back tilt, which we were able to calculate the angle of both using two equations: $\text{roll} = \text{asin}(g_x)$, $\text{pitch} = \text{asin}(g_z)$. The radian values computed (ranging from $-\pi/2$ to $\pi/2$), were then scaled to fit in a 80-character wide terminal. So that $\pm\pi/2$ corresponded with to ± 39 characters on the screen, making it so the center column 40 was the center and represented neutral orientation. Positive values printed out the character 'r' to the right of the center, while negative displayed 'l' to the left.

Design

Designing my code was first based on the skeleton file that was provided with the assignment. It had multiple functions that we had to build ourselves, the `read_line()` function, held the `scanf()`, and continuously grabbed the input until Square was pressed. The `roll()` and `pitch()` function did the actual calculations for the angle using the gyroscope values, while clamping the inputs between -1 and 1 to prevent errors. The `scaleRadsForScreen()` function converted the radian angle values into scaled integers to fit between -39 and 39 to match the width of the screen. The `print_chars()` function held the only `printf()`, and printed a specific character. The `graph_line()` function built the entire bar for each from, doing so by using the `print_chars()` function, placing 0 in the center, and filling either side with 'r' or 'l' characters. The main loop, called upon the `read_line()` function continuously in order to gather the input information, determine which mode it was in based on button presses, computed the angle, scaled it to fit, and then put the result into `graph_line()`. The output scrolled downwards, filling the graph with new lines.

Testing

Testing involved using the pre-provided ds4rd.exe programming and compiling my own program and running the two together with the required flags. First test showed that the roll and pitch values did change with the controller as it moved, showing there were no underlying issues with the controller. I also had to figure out the order of inputs of the buttons, because the function provided had them in the incorrect order. To test this I looked at only the ds4rd.exe programming, using the -b flag to look at button data. Then just pressing the buttons find the order. I tested the scale and the graph by tilting the controller to it's extremes, so that the whole line would be filled, as well as at level , so that it only out put 0.

Comments

This lab helped me a lot with creating a simple display, using code that I wouldn't have thought of at first to do so. The biggest challenge that I faced when working on this lab, was the unreliability of controllers in the lab. Many of the controllers were broken, some of the buttons would be in a constant state of being pressed. This made it so that when I would run my code, the square button would end the program when I didn't want it to. This made it frustrating to work, as I didn't realize it at first, and I thought there was just another larger issue with my program. So I would try and fix my code when it didn't need to be changed, I was only able to fix it after I look at the raw controller data of the buttons and noticed that it would provide input when I didn't press anything.

Questions and Experiments

1. How did you scale your values? Write an equation and justify it.

Displaying the angle on a specifically 80-character screen, was down by converting the angles range, to that of -39 to 39 characters. The equation used to scale is: $\text{scaled_value} = (39 / (\pi / 2)) \times \text{angle_radians}$. This equation made it so that when the controller reaches the maximum tilt of $\pm\pi/2$ radians, the graph reaches the edges of the screen with 39 characters on either side. It also scaled smaller values of tilt, proportionally to the length of the bar.

2. How many degrees does each letter in your graph represent? What do you notice near the limits?

Because the range of $\pm 90^\circ$ (radians $-\pi/2$ to $+\pi/2$) is represented by 39 characters on each side, each letter represents about 2.31° of tilt, simply by dividing $90 / 39$. When testing, I noticed that the graph seems to become less responsive at its extreme values. I assume this is because the sine function approaches its limit of plus or minus 1. Since it never approaches its limit, it seems to behave differently than in its normal range.

Implementation

```
// 185 lab6.c

//

// This is the outline for your program

// Please implement the functions given by the prototypes below and

// complete the main function to make the program complete.

// You must implement the functions which are prototyped below exactly

// as they are requested.


#include <stdio.h>

#include <math.h>

#define PI 3.141592653589


//NO GLOBAL VARIABLES ALLOWED


//PRE: Arguments must point to double variables or int variables as appropriate

//This function scans a line of DS4 data, and returns

// True when the square button is pressed

// False Otherwise

//This function is the ONLY place scanf is allowed to be used

//POST: it modifies its arguments to return values read from the input line.

int read_line(double* g_x, double* g_y, double* g_z, int* Button_T, int* Button_C, int*

Button_X, int* Button_S);


// PRE: -1.0 <= x_mag <= 1.0

// This function computes the roll of the DS4 in radians

// if x_mag outside of -1 to 1, treat it as if it were -1 or 1
```

```
// POST:  $-\pi/2 \leq \text{return value} \leq \pi/2$ 
```

```
double roll(double x_mag);
```

```
// PRE:  $-1.0 \leq y\_mag \leq 1.0$ 
```

```
// This function computes the pitch of the DS4 in radians
```

```
// if y_mag outside of -1 to 1, treat it as if it were -1 or 1
```

```
// POST:  $-\pi/2 \leq \text{return value} \leq \pi/2$ 
```

```
double pitch(double y_mag);
```

```
// PRE:  $-\pi/2 \leq \text{rad} \leq \pi/2$ 
```

```
// This function scales the roll value to fit on the screen
```

```
// POST:  $-39 \leq \text{return value} \leq 39$ 
```

```
int scaleRadsForScreen(double rad);
```

```
// PRE:  $\text{num} \geq 0$ 
```

```
// This function prints the character use to the screen num times
```

```
// This function is the ONLY place printf is allowed to be used
```

```
// POST: nothing is returned, but use has been printed num times
```

```
void print_chars(int num, char use);
```

```
//PRE:  $-39 \leq \text{number} \leq 39$ 
```

```
// Uses print_chars to graph a number from -39 to 39 on the screen.
```

```
// You may assume that the screen is 80 characters wide.
```

```
void graph_line(int number);
```

```
int main() {
```

```

double x, y, z;

int time, b_Triangle, b_X, b_Square, b_Circle;

double roll_rad, pitch_rad;

int scaled_value;

int mode = 0; // 0 = roll, 1 = pitch

int counter = 0;

while (read_line(&x, &y, &z, &b_Triangle, &b_Circle, &b_X, &b_Square)) {
    if (b_Triangle) {
        counter++;

        if(counter % 2 == 0) {
            mode = 0;
        }
        else {
            mode = 1;
        }
    }
}

if (b_Square) break;

if (mode == 0)
    roll_rad = roll(x), scaled_value = scaleRadsForScreen(roll_rad);
else
    pitch_rad = pitch(z), scaled_value = scaleRadsForScreen(pitch_rad);

graph_line(scaled_value);

fflush(stdout);

```

```
}
```

```
return 0;
```

```
}
```

```
int read_line(double* g_x, double* g_y, double* g_z, int* Button_T, int* Button_C, int*  
Button_X, int* Button_S) {
```

```
    int result = scanf("%lf,%lf,%lf,%d,%d,%d,%d,%d", g_x, g_y, g_z, Button_T, Button_C,  
Button_X, Button_S);
```

```
}
```

```
double roll(double x_mag) {
```

```
    if (x_mag > 1.0) x_mag = 1.0;
```

```
    if (x_mag < -1.0) x_mag = -1.0;
```

```
    return asin(x_mag);
```

```
}
```

```
double pitch(double y_mag) {
```

```
    if (y_mag > 1.0) y_mag = 1.0;
```

```
    if (y_mag < -1.0) y_mag = -1.0;
```

```
    return asin(y_mag);
```

```
}
```

```
int scaleRadsForScreen(double rad) {
```

```
    // Map  $[-\pi/2, \pi/2] \rightarrow [-39, 39]$ 
```

```
    double scale = 39.0 / (PI / 2.0);
```

```
    return (int)(rad * scale);
```

```
}
```

```
void print_chars(int num, char use) {
```

```
    for (int i = 0; i < num; i++) {
```

```
        printf("%c", use);
```

```
    }
```

```
}
```

```
void graph_line(int number) {
```

```
    if (number == 0) {
```

```
        print_chars(39, ' ');
```

```
        printf("0\n");
```

```
    } else if (number > 0) {
```

```
        print_chars(39, ' ');
```

```
        printf("0");
```

```
        print_chars(number, 'r');
```

```
        printf("\n");
```

```
    } else { // number < 0
```

```
        print_chars(39 + number, ' ');
```

```
        print_chars(-number, 'l');
```

```
        printf("0\n");
```

```
    }
```

```
}
```